Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»

Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки

ЗВІТ

з навчального проекту №1
з дисципліни «Методи та технології паралельного програмування»

Тема: «Моделювання броунівського руху засобами мультипоточного програмування»

Варіант №4

Виконав:
студент 1 курсу, групи ІМ-51мн
Ковальов Олександр

Перевірив:
асистент, Лесик Валентин Олександрович

Дата здачі: 30.04.2026

КИЇВ – 2026

Мета роботи. Освоїти моделі паралелізму. Навчитись працювати з паралельними симуляціями. Дослідити проблеми синхронізації та спільного доступу до даних. Проаналізувати продуктивність запропонованого підходу.

Опис прикладної задачі. Розглядається процес броунівського руху частинок домішки у двовимірному кристалі, представленому як сітка розміром $M * N$. У кристалі розміщено K частинок, які виконують випадкове переміщення між сусідніми клітинками (вгору, вниз, вліво, вправо) з певними ймовірностями. Основні правила переходу частинок: на кожному кроці часу частинка обирає напрямок руху випадково; якщо частинка досягає межі і рухається в сторону межі, то вона відбивається, тобто лишається на місці (або застосовується інша стратегія). У кожній клітинці може бути кілька частинок, але загальна кількість частинок зберігається.

Завдання. Потрібно змоделювати поведінку частинок у двовимірному кристалі засобами мультипоточного програмування), використовуючи для кожного атома домішки свій потік виконання. Результатом роботи моделювання має бути набір «моментальних знімків», що характеризують стан розподілу частинок домішки по клітинах кристалу від початку до кінця терміну моделювання із заданою дискретністю кроку відображення стану.

Кожен потік моделювання частинки у кожному акті моделювання має згенерувати випадкове число, за яким прийняти рішення про перехід до наступної клітинки за заданими ймовірностями. Під час роботи програми виникають потенційні проблеми, що пов'язані з особливостями взаємодії потоків та спільного доступу до даних. Необхідно продемонструвати наявність цих проблем, запропонувати рішення та показати як саме це рішення долає проблему.

Мова програмування, фреймворки та допоміжні елементи (наприклад для забезпечення візуалізації) обираються на власний розсуд студента.

З потенційних проблем можна виділити:

- Кілька потоків змінюють одну клітинку, що призводить до втрати частинок або некоректні значення, тобто проблема `race condition`.
- `Deadlock`: потоки блокують один одного.
- Перевантаження потоками: один потік на одну частинку погано масштабується.
- Генерація випадкових чисел: доступ до генератора випадкових чисел є вузьким місцем.
- Відсутність консистентності знімків: стан кристалу читається під час моди-

фікації.

Потенційні рішення:

- Застосування `mutex`, `lock`, `atomic` операцій.
- Глобальний порядок захоплення блокувань.
- Використання `lock-free` структур, `task pool`, `chunking`, `barrier`, `immutable snapshot`.

Приклад виконання проєкту:

1. Задається сітка кристалу, кількість атомів та шанси переходу атомів.
2. Задається час симуляції та дискретизація отримання знімків.
3. Кожному атому домішки відповідає потік виконання, який містить в собі індекс атома домішки. Робота потоку полягає в модифікації масиву кристалу і зміни свого індексу.
4. Симуляція запускається, отримує задану кількість знімків, зупиняється.
5. В кінці порівнюється число домішок до початку симуляції та після.

Студенти можуть запропонувати власну реалізацію проєкту, методів або елементів програми чи завдання, якщо вони краще демонструють проблеми із використанням багатопоточності, або якщо ці нововведення вирішують поставлену задачу симуляції краще. Всі пропозиції та девіації від вимог необхідно описувати та обґрунтовувати.

Програма повинна реалізовувати процес симуляції руху атомів в кристалі, робити моментальні зліпки стану кристалу та демонструвати наявні проблеми багатопоточності та їх вирішення. В кінці, програма повинна підрахувати кількість домішок в кристалі і вивести результат.

Для спрощення процесу захисту можна реалізувати лише частину проекту, або повністю весь проект для отримання максимального балу. Часткове рішення не передбачає візуалізації та двовимірного кристалу. Повноцінне рішення повинно включати одне із вирішень складних проблем.

Повноцінне виконання включає в себе реалізацію симуляції броунівського руху з візуалізацією в двовимірному кристалі, правильний механізм отримання зліпків стану кристалу, демонстрацію виникнення `deadlock` та `race condition`, задання `seed` та забезпечення відтворюваності симуляції (це передбачає надання багатопотокового рішення інтеграції випадкових чисел в процес вибору руху частинок) забезпечення отримання точного зліпку без зупинки симуляції, забезпечення процесу масштабування для атомів, тобто вирішення проблеми 1 атом = 1 потік. Також, треба провести комплексний аналіз результатів, впливу розподілу ресурсів одного потоку на кожну частинку, вибору початкових параметрів, про-

блем паралелізму, що виникли та описати запропоновані варіанти покращення чи вирішення зафіксованих проблем.

Часткове виконання проекту включає в себе реалізацію симуляції броунівського руху з візуалізацією в двовимірному кристалі, правильний механізм отримання зліпків стану кристалу, демонстрація виникнення `deadlock` та `race condition`, проведення базового аналізу результатів.

Мінімальний проект повинен мати симуляцію броунівського руху з візуалізацією в одновимірному кристалі (вистачить консольного додатку); достатньо показати проблему із спільним доступом до даних з синхронізацією та без синхронізації.

Хід роботи.

Вступ та опис предметної області. Предметом дослідження у даному проекті є процес броунівського руху частинок домішки у двовимірному кристалі. Фізична модель представлена у вигляді дискретної сітки розміром $M \times N$, де кожна комірка може містити довільну кількість атомів домішки. Основною задачею є моделювання хаотичного переміщення частинок між сусідніми вузлами сітки за заданими ймовірностями (вгору, вниз, вліво, вправо) у багатопотоковому середовищі.

Обрані інструменти та технології.

Для реалізації проекту було обрано сучасний технологічний стек, що дозволяє поєднати високу продуктивність із гарантіями безпеки пам'яті:

- **Мова програмування Rust:** обрана завдяки її моделі володіння (*ownership*) та суворим перевіркам під час компіляції, що унеможливлює виникнення помилок сегментації пам'яті та більшості проблем синхронізації (*data races*) ще на етапі розробки.
- **Бібліотека Rayon:** використана для реалізації паралелізму даних. Замість наївної моделі «один атом = один потік ОС», Rayon використовує ефективний механізм крадіжки завдань (*work-stealing*) у пулі потоків (*thread pool*), що дозволяє масштабувати симуляцію до 100,000+ атомів без значних накладних витрат.
- **Фреймворк egui:** легковажна бібліотека для створення графічного інтерфейсу в режимі негайного малювання (*immediate mode GUI*). Вона дозволяє реалізувати плавну візуалізацію руху часток у реальному часі та забезпечує

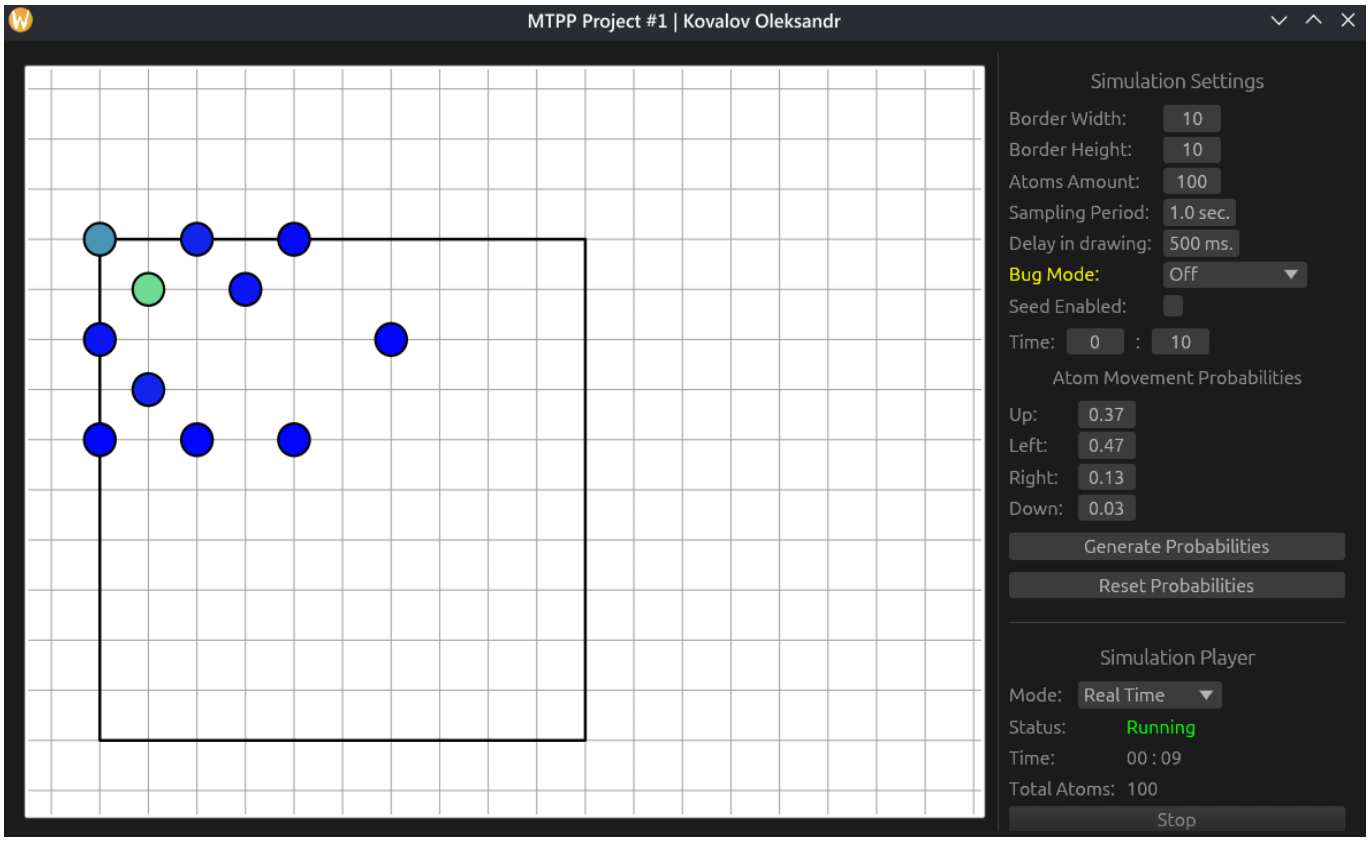
високу інтерактивність налаштувань.

Архітектура застосунку.

Архітектура програмного комплексу побудована на принципі розподілу обов'язків (*Separation of Concerns*) та складається з двох основних модулів, що працюють асинхронно:

1. **Backend (Обчислювальне ядро):** працює в окремому фоновому потоці. Модуль `Engine` відповідає за життєвий цикл симуляції, виконання ітерацій (*ticks*) та генерацію зліпків стану (*snapshots*). Для обміну даними з графічним потоком використовуються швидкі асинхронні канали (*crossbeam-channel*).
2. **Frontend (Інтерфейс та візуалізація):** головний потік програми, який керує циклом рендерингу. Він приймає команди від користувача, відправляє їх у ядро та відображає отримані зліпки стану за допомогою компонентів `CanvasComponent` та `SimulationVisualizer`.

Особливістю архітектури є використання `AtomicUsize` для представлення клітинок кристалу. Це дозволяє реалізувати *lock-free* підхід до оновлення даних: потоки Rayon можуть одночасно змінювати кількість часток у різних комірках без використання важких блокувань (*mutexes*), що мінімізує час очікування потоків та максимізує утилізацію ядер процесора.



Аналіз проблеми масштабування: «1 атом = 1 потік».. Однією з найпоширеніших помилок при проектуванні паралельних систем є використання наївного підходу до масштабування, за якого кожній незалежній сутності (у нашому випадку – атому) виділяється окремий потік операційної системи (OS thread). Хоча концептуально така модель виглядає логічною, на практиці вона призводить до катастрофічного падіння продуктивності або повного краху програми при спробі збільшити кількість частинок до десятків чи сотень тисяч.

Проблема накладних витрат операційної системи.

Виділення окремого потоку під кожен атом створює два критичні вузькі місця на рівні операційної системи:

- **Вичерпання пам'яті (Stack Size):** Кожен потік в ОС (Linux/Windows) при створенні отримує власний стек, розмір якого за замовчуванням становить від 2 МБ до 8 МБ. При спробі симуляції 100,000 атомів, операційній системі необхідно виділити близько 800 ГБ віртуальної пам'яті лише під стеки потоків. Навіть з урахуванням механізму *memory overcommit*, це швидко призводить до вичерпання системних ресурсів.
- **Перемикання контексту (Context Switching):** Сучасні процесори мають обмежену кількість апаратних потоків (наприклад, 16 або 32). Якщо активних програмних потоків стає 100,000, планувальник ОС (Scheduler) змушений постійно призупиняти одні потоки і запускати інші. Процес збереження та відновлення регістрів процесора і таблиць сторінок пам'яті займає більше процесорного часу, ніж сам розрахунок переміщення атома на сітці. Цей стан системи відомий як *thrashing* (пробуксовка).

Демонстрація проблеми (Bug Mode).

Для практичної перевірки цієї гіпотези в розробленому програмному забезпеченні реалізовано спеціальний режим тестування `Bug Mode: 1 Atom = 1 Thread`.

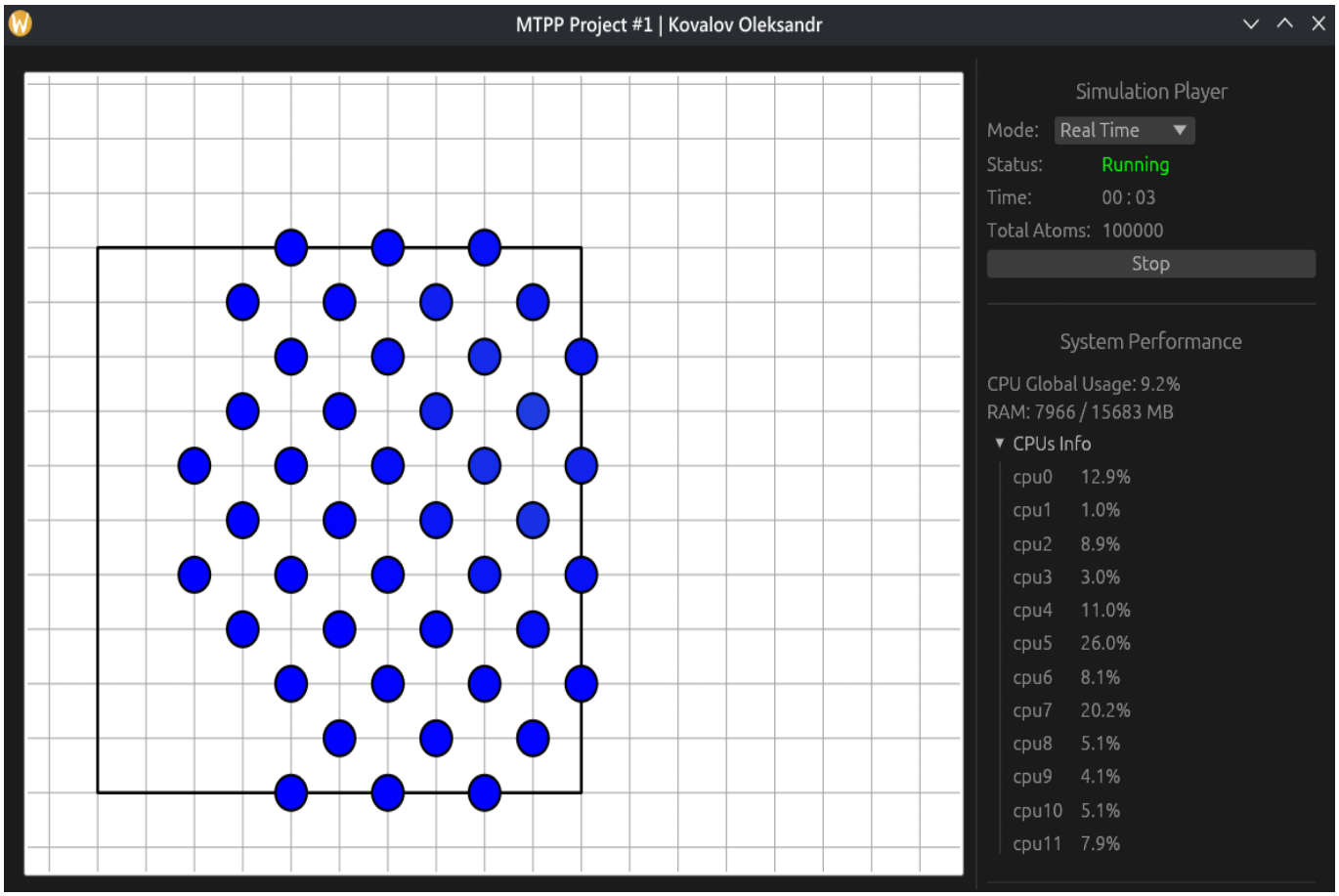
У цьому режимі для кожного атома примусово створюється окремий фізичний потік. Як і очікувалося, при спробі запуску симуляції на великій кількості атомів, операційна система відмовляє у виділенні ресурсів, або ж починається перевантаження.

Обґрунтування використання пулу потоків (Rayon).

Для вирішення проблеми масштабованості архітектуру обчислювального ядра було змінено на користь **пулу потоків (Thread Pool)**. У проекті інтегровано бібліотеку `Rayon`, яка реалізує паттерн паралелізму даних на основі алгоритму крадіжки завдань (*work-stealing*).

1. При старті рушія Rayon створює фіксовану кількість робочих потоків (*worker threads*), яка точно дорівнює кількості логічних ядер центрального процесора (наприклад, 12).
2. Замість створення потоку під кожен атом, 100,000 ітерацій циклу обробки атомів розбиваються на менші блоки і розподіляються між цими 12 робочими потоками.
3. Якщо один з потоків закінчує обробку свого блоку швидше, він «краде» частину нерозподіленої роботи в іншого потоку, забезпечуючи ідеальне балансування навантаження (*load balancing*).

Такий підхід дозволив повністю усунути накладні витрати на *Context Switching* та виділення пам'яті під стеки. Як видно з панелі моніторингу ресурсів у застосунку, симуляція 100,000 атомів за допомогою Rayon споживає мінімум оперативної пам'яті і стабільно утилізує CPU без зависань.



Забезпечення відтворюваності симуляції та генерація випадкових чисел. Відтворюваність є критичною вимогою для наукових симуляцій та систем моделювання. Вона дозволяє досліднику повторно запустити експеримент із тими самими вхідними параметрами та отримати ідентичний результат. У контексті броунівського руху це означає, що траєкторія кожної частинки має бути детермінованою при заданому початковому значенні (*seed*).

Проблема глобальних ГВЧ у багатопотоковому середовищі.

Типовим підходом в однопотокових програмах є використання одного глобального генератора випадкових чисел (ГВЧ). Проте при переході до багатопоточності цей підхід створює дві фундаментальні проблеми:

1. **Деградація продуктивності:** ГВЧ – це об'єкт із внутрішнім станом, який змінюється після кожної генерації числа. Для безпечного доступу з багатьох потоків глобальний ГВЧ має бути захищений примітивом синхронізації (наприклад, `Mutex` або `RwLock`). У симуляції з 100,000 атомів, де кожен атом потребує випадкове число на кожному кроці, потоки будуть витрачати більшість часу в черзі на захоплення м'ютекса, що нівелює переваги паралелізму.
2. **Втрата детермінованості:** Навіть якщо глобальний ГВЧ ініціалізований конкретним *seed*, порядок, у якому потоки отримують доступ до нього, залежить від планувальника ОС. Якщо Потік А затримається на мікросекунду, Потік Б першим візьме наступне число з послідовності. В результаті частинки отримають різні вектори руху в різних запусках симуляції, попри однаковий *seed*.

Рішення: Локальні генератори та Lock-free підхід.

Для вирішення описаних проблем у проекті було застосовано стратегію **локалізації стану генератора**. Кожна структура `Atom` містить власний примірник швидкого некриптографічного генератора `SmallRng`.

- **Ініціалізація:** При старті симуляції головний потік приймає загальний *seed*. На його основі для кожного атома генерується унікальний, але детермінований локальний *seed* (наприклад, поєднання базового числа та ідентифікатора атома).
- **Незалежність обчислень:** Під час виконання ітерації (`tick`), кожен робочий потік у пулі Rayon ексклюзивно працює зі своїм набором ато-

мів. Генерація випадкового числа відбувається всередині структури атома: `atom.rng.gen::<f64>()`.

Переваги обраного підходу

Застосована архітектура забезпечує:

1. **Абсолютну відтворюваність:** Оскільки послідовність чисел для кожного атома залежить тільки від його власного стану, симуляція видає ідентичний результат незалежно від кількості ядер процесора або навантаження на ОС.
2. **Максимальну швидкість (Lock-free):** Потоки не мають спільних даних для генерації чисел, що повністю усуває витрати на синхронізацію.
3. **Ефективність кешу:** Стан ГВЧ знаходиться у пам'яті поруч із координатами атома, що покращує локальність даних та зменшує кількість промахів кешу (*cache misses*).

Дослідження проблем синхронізації даних. Розробка багатопотокових систем, де кілька потоків одночасно модифікують спільний стан пам'яті (у нашому випадку – двовимірну сітку кристалу), вимагає особливої уваги до синхронізації. Неправильне керування спільним доступом неминуче призводить до класичних проблем паралелізму. Для дослідження цих аномалій у проекті було реалізовано керовані режими помилок (*Bug Modes*).

Стан гонитви (Race Condition).

Стан гонитви виникає, коли результат виконання програми залежить від непередбачуваного порядку планування потоків операційною системою. У симуляції броунівського руху критичною секцією є оновлення кількості атомів у комірках: потоку необхідно зменшити лічильник у старій комірці та збільшити у новій.

Демонстрація проблеми: У режимі `BugMode::RaceCondition` атомарну операцію було штучно розділено на дві незалежні дії: читання (`load`) та запис (`store`). Виникає класична проблема «втраченого оновлення» (*Lost Update*):

1. Потік A та Потік B одночасно намагаються перемістити свої атоми у комірку X, яка наразі містить 5 атомів.
2. Потік A зчитує значення 5. Майже одночасно Потік B також зчитує значення 5.
3. Потік A додає 1 і записує 6. Потік B також додає 1 і затирає результат Потоку A, знову записуючи 6.

4. Хоча в комірку прибуло два атоми, лічильник збільшився лише на один. Один атом «зник» із системи.

Як наслідок, під час роботи цього режиму загальна кількість атомів у системі (*Total Atoms*), яка має бути константою (наприклад, 100 000), починає стрімко і хаотично падати або зростати, руйнуючи фізичну достовірність симуляції.

Вирішення (Lock-free архітектура): Для усунення цієї проблеми в основному (безпечному) режимі використовуються атомарні типи даних `AtomicUsize`. Замість пари `load/store` застосовуються методи `fetch_add` та `fetch_sub`. Ці функції компілюються у спеціальні апаратні інструкції процесора (наприклад, префікс `LOCK` в архітектурі x86), які гарантують, що цикл «читання-модифікація-запис» (*Read-Modify-Write*) є неподільним. Використання моделі пам'яті `Ordering::Relaxed` є достатнім і оптимальним, оскільки нам важлива лише атомарність самої операції над конкретним лічильником, а не синхронізація інших ділянок пам'яті.

Взаємне блокування (Deadlock).

Взаємне блокування (*Deadlock*) – це стан, у якому два або більше потоків назавжди зависають, очікуючи на ресурси, які утримуються іншими потоками з цієї ж групи.

Штучне відтворення: Для демонстрації цієї проблеми в режимі `BugMode::Deadlock` було створено два глобальні примітиви синхронізації – м'ютекси A та B. Алгоритм штучно провокує виконання умови «Кругового очікування» (*Circular Wait*, одна з чотирьох умов Коффмана):

- Половина потоків (атоми з парними координатами) захоплюють м'ютекс A, виконують мікрозатримку, після чого намагаються захопити м'ютекс B.
- Інша половина потоків (атоми з непарними координатами) діють дзеркально: захоплюють B, а потім намагаються отримати A.

У результаті Потік 1 утримує A і чекає на B, тоді як Потік 2 утримує B і чекає на A. Симуляція повністю паралізується: розрахунки припиняються, час у застосунку зупиняється, а процесор переходить у стан простою.

Обґрунтування переваг lock-free підходу:

Головною перевагою поточної архітектури симулятора є повна відсутність м'ютексів (`Mutex`) чи блокувань читання-запису (`RwLock`) на критичних шляхах виконання (*hot paths*). Оскільки синхронізація поля відбувається через неблокуючі атомарні операції, а генерація випадкових чисел ізольована локально в кожному

атомі, виникнення *Deadlock* у безпечному режимі математично неможливе. Потоки ніколи не очікують один на одного, що забезпечує лінійне масштабування продуктивності.

Механізм отримання зліпків стану (Snapshots). Для аналізу розподілу частинок у кристалі з часом, система повинна мати можливість зберігати проміжні стани симуляції – «моментальні зліпки» (*snapshots*). Відповідно до вимог складного рівня проекту, процес зняття зліпку не повинен зупиняти або блокувати основний обчислювальний алгоритм.

Неблокуюче отримання зліпку пам'яті.

У класичних багатопотокових системах для отримання консистентного зліпку стану необхідно призупинити всі робочі потоки (наприклад, за допомогою глобального `RwLock`), зробити копію даних і лише потім дозволити потокам продовжити роботу. Це створює суттєві затримки та знижує загальну пропускну здатність системи.

У розробленому симуляторі масив кристалу представлений як `Vec<AtomicUsize>`. Отримання зліпку реалізовано всередині фонового рушія (`Engine`) шляхом лінійного проходу по масиву та зчитування атомарних значень:

```
let field: Vec<usize> = crystal.field.iter()
.map(|c| c.load(std::sync::atomic::Ordering::Relaxed))
.collect();
```

Переваги використання `Ordering::Relaxed`: При використанні цього типу впорядкування пам'яті процесор не вставляє бар'єри пам'яті (*memory fences*). Операція зчитування (`load`) виконується як звичайна машинна інструкція, але з гарантією атомарності на рівні окремо взятого лічильника. Завдяки цьому:

1. Процес створення зліпку не накладає жодних глобальних блокувань (*Locks*). Робочі потоки `Rayon` продовжують вільно переміщувати атоми та модифікувати масив паралельно з процесом читання.
2. Зчитування відбувається на максимальній швидкості оперативної пам'яті.
3. Даний підхід ідеально відповідає вимозі «Забезпечення отримання точного зліпку без зупинки симуляції», мінімізуючи вплив спостерігача на систему (*Observer Effect*).

Збереження історії (Snapshot Storage).

Для забезпечення можливості постаналізу, у стан графічного інтерфейсу інтегровано структуру `SnapshotStorage`. Згідно із заданим у налаштуваннях періодом дискретизації (*Sampling Period*), обчислювальне ядро періодично генерує та надсилає через `crossbeam`-канал нові зліпки.

Структура `SnapshotStorage` зберігає ці кадри у вигляді вектора і надає користувачу інтерфейс для навігації. Після завершення симуляції користувач може перемкнути режим перегляду на `Snapshot Mode` і переміщуватися в часі, досліджуючи покадровий розподіл атомів без необхідності повторного перезапуску розрахунків.

Візуалізація та відображення щільності (Heatmap).

Кожен зліпок (`CrystalSnapshot`) трансформується у набір графічних примітивів для відмальовування на полотні (`CanvasComponent`) за допомогою фреймворку `egui`.

Для ефективного візуального аналізу скупчень частинок застосовується метод кольорового мапування (*Heatmap*).

Колір кожної клітинки на екрані розраховується на основі відсоткової частки атомів у цій клітинці відносно загальної кількості частинок у системі. Функція `coloring` плавно змінює RGB-канали:

- **Синій колір:** мінімальна кількість атомів у комірці (близько до 0%).
- **Зелений/Жовтий колір:** середнє накопичення (близько 50%).
- **Червоний колір:** критичне скупчення (велика концентрація домішки в одному вузлі).

Додатково реалізовано оптимізовану систему підказок (*Tooltips*): при наведенні курсору на певну клітинку, завдяки однозначній системі ідентифікаторів на основі індексу масиву (`cell_index`), користувач отримує точну інформацію про координати вузла та абсолютну кількість атомів у ньому.

Аналіз впливу початкових параметрів та поведінки системи. Модель броунівського руху, реалізована у даному програмному комплексі, є стохастичною системою, макроскопічний стан якої повністю визначається набором мікроскопічних початкових параметрів: розподілом ймовірностей руху та правилами обробки зіткнень із межами кристалу. Для дослідження цих процесів використовується вбудована система візуалізації щільності (*Heatmap*).

Вплив розподілу ймовірностей (Спрямований дрейф).

У налаштуваннях симуляції (`SimulationSettings`) користувач може задавати чотири ймовірності напрямку руху: P_{up}, P_{down}, P_{left}, P_{right}, сума яких має дорівнювати 1.0. Розподіл цих значень формує два різні фізичні сценарії:

1. **Ізотропна дифузія (Рівномірний розподіл):** Якщо всі ймовірності однакові (по 0.25), рух атомів є класичним випадковим блуканням (*Random Walk*) без виділеного напрямку. У такому режимі частинки рівномірно розподіляються по всьому об'єму кристалу. Візуально на тепловій карті переважають сині та світло-зелені відтінки, оскільки жодна з комірок не накопичує критичної маси атомів.
2. **Анізотропний рух (Наявність зовнішньої сили):** Якщо одна з ймовірностей суттєво переважає (наприклад, $P_{up} = 0.80$, а інші по $0.05..0.10$), це симулює наявність зовнішнього поля (наприклад, гравітаційного, електричного або потоку рідини/газу). Виникає ефект макроскопічного дрейфу: уся хмара атомів починає цілеспрямовано мігрувати у заданому напрямку.

Роль граничних умов та формування скупчень.

Межі двовимірного кристалу відіграють ключову роль у формуванні фінального розподілу частинок. У процесі розробки обчислювального ядра було реалізовано алгоритм точного позиціювання на межах (*Boundary Clamp*): коли обчислені нові координати частинки виходять за межі розміру кристалу ($0 \leq x \leq$ width, $0 \leq y \leq$ height), частинка не генерує помилку індексації, а залишається на крайній доступній лінії сітки і відскакує.

Взаємодія спрямованого дрейфу (анізотропних ймовірностей) та граничних умов призводить до формування щільних скупчень:

- Якщо задано рух переважно вгору, частинки швидко досягають верхньої межі кристалу.
- Оскільки подальший рух у цьому напрямку заблоковано граничною умовою, атоми починають акумулюватися у верхніх комірках і відскакувати.
- Локальні ймовірності руху вліво/вправо дозволяють частинкам продовжувати ковзати вздовж межі, поступово заповнюючи весь крайній ряд кристалу.

Візуальний аналіз за допомогою Heatmap-системи.

Ефективність виявлення подібних аномалій або скупчень досягається завдяки алгоритму градієнтного фарбування. У методі `CrystalSnapshot::coloring` розраховується інтенсивність (*intensity*) як відношення кількості атомів у конкретній клітинці до їхньої загальної кількості у системі.

Цей показник нормується і перетворюється на RGB-градієнт:

```
let r = (255.0 * intensity) as u8;
let b = (255.0 * (1.0 - intensity)) as u8;
let g = (255.0 * (1.0 - (intensity * 2.0 - 1.0).abs())) as u8;
```

Завдяки цьому дослідник миттєво ідентифікує макроскопічний стан системи: поява яскраво-червоних точок на екрані безпомилково свідчить про те, що значний відсоток усіх наявних частинок зібрався в одному вузлі простору. Це перетворює застосунок із простого симулятора мікрочастинок на повноцінний інструмент теплофізичного моделювання.

Висновок. У ході виконання навчального проекту було розроблено програмний комплекс для моделювання броунівського руху часток у двовимірному кристалі з використанням сучасних методів паралельного програмування на мові Rust.

На основі проведеного аналізу та результатів тестування можна зробити наступні висновки:

1. **Ефективність моделей паралелізму:** Доведено, що наївна модель «1 атом = 1 потік ОС» не є життєздатною для систем із великою кількістю активних об'єктів (100,000+). Використання пулу потоків на основі алгоритму *work-stealing* (бібліотека Rayon) дозволило досягти стабільної продуктивності та уникнути критичних накладних витрат на перемикання контексту та виділення пам'яті під стеки.
2. **Переваги Lock-free синхронізації:** Застосування атомарних операцій (`AtomicUsize`) замість класичних м'ютексів дозволило повністю усунути можливість виникнення взаємних блокувань (*Deadlocks*) у робочому режимі та мінімізувати простої потоків. Це забезпечило високу пропускну здатність обчислювального ядра навіть при інтенсивному доступі до спільних даних.
3. **Забезпечення детермінованості:** Розподіл локальних генераторів випадкових чисел (`SmallRng`) усередину структур атомів дозволив реалізувати повну відтворюваність симуляції за початковим значенням (*Seed*), що є критичним для верифікації наукових моделей.

4. **Аналіз аномалій:** Реалізовані режими демонстрації помилок (*Race Condition, Deadlock*) наочно продемонстрували наслідки некоректної синхронізації, такі як втрата консистентності даних (втрачені оновлення) та повна зупинка системи.

Розроблений інструментарій візуалізації (теплові карти та система зліпків) підтвердив адекватність побудованої фізичної моделі: за умови анізотропних ймовірностей спостерігався чіткий дрейф часток та їх акумуляція на межах кристалу, що відповідає реальним фізичним процесам дифузії в обмеженому об’ємі.

Таким чином, обрана архітектура на базі Rust та Rayon продемонструвала високу масштабованість, безпеку та продуктивність, повністю виконуючи поставлені вимоги проекту.